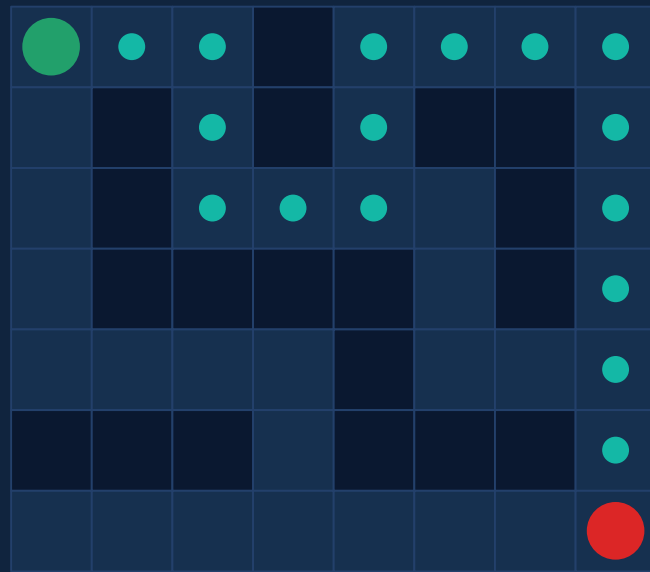


ROBOTICS TRAINEE PROGRAMME · THIS WEEK

Pathfinding on Occupancy Grids

From a floor-plan image to an A* path a real robot can drive — maps, graphs, heuristics, code, and weekend build tasks.

Lecture pack · Intermediate track · Includes simulation + physical weekend practicals





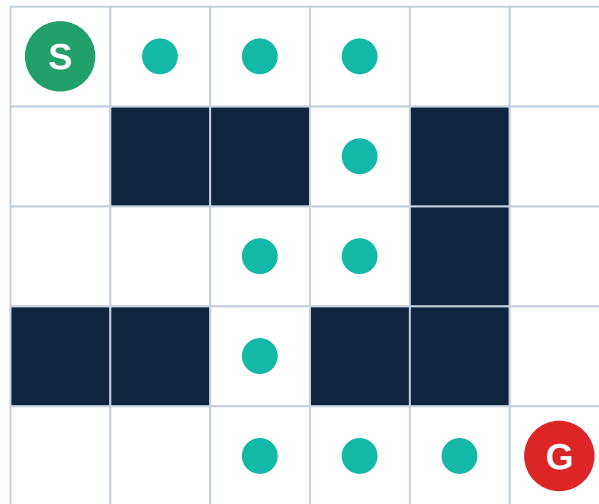
Why Pathfinding?

Your robot has a map of the warehouse. Shelves, walls, one charging dock. The question every mobile robot must answer, every second:

“Given a map with obstacles, find a collision-free — ideally shortest — route from start to goal.”

Where you will meet this problem

- Warehouse robots — picking routes between shelves
- Game AI — NPCs chasing or fleeing the player
- Self-driving cars — lane-level route planning
- Evacuation planning — safest exits from buildings
- Drones — obstacle-aware waypoint missions



A collision-free route on a 6 × 5 occupancy grid



Representing the World: Occupancy Grids

An occupancy grid is a 2D matrix laid over the floor plan — graph paper over the world. Every cell holds one of two values:

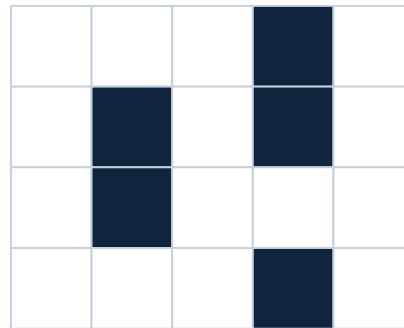
1

FREE — the robot may pass

0

OCCUPIED — obstacle, blocked

```
grid = np.array([
    [1, 1, 1, 0, 1],
    [1, 0, 1, 0, 1],
    [1, 0, 1, 1, 1],
    [1, 1, 1, 0, 1],
    ])
```



Same data, two views

Why this representation wins: it is exactly what a NumPy array is. Sensors (LiDAR, cameras) can write into it, and search algorithms can read from it — one data structure connects perception to planning.



From Image to Grid: The Pipeline

Real maps arrive as images. Three steps turn pixels into a matrix the planner can search:

1 · GRAYSCALE

Load the image so each pixel is one number, 0 (black) to 255 (white).

```
cv2.imread(path,  
cv2.IMREAD_GRAYSCALE)
```

2 · THRESHOLD

Pick a cutoff. Above it → 255 (free). Below it → 0 (blocked).

```
cv2.threshold(img, 128,  
255, cv2.THRESH_BINARY)
```

3 · NORMALIZE

Divide by 255 so every cell is exactly 0 or 1. Done — occupancy grid.

```
grid = thresh // 255
```



Threshold tuning: lower cutoff = more free space (permissive); higher = less free space (conservative). 128 is the midpoint default — tune it per map.



The Conversion Code

```
import cv2
import numpy as np

def create_occupancy_grid_from_image(image_path,
threshold=128):
    # Load as grayscale: each pixel is 0-255
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Threshold: pixels > 128 become 255 (free), rest 0
    # (blocked)
    _, thresholded = cv2.threshold(image, threshold, 255,
                                  cv2.THRESH_BINARY)

    # Normalize: 255 -> 1, 0 -> 0
    occupancy_grid = thresholded // 255

    return occupancy_grid, image
```

Two functions to know

`cv2.imread(...)`

Loads the image straight into a NumPy 2D array.

`cv2.threshold(...)`

Binary thresholding: one cutoff, two worlds — free or blocked.



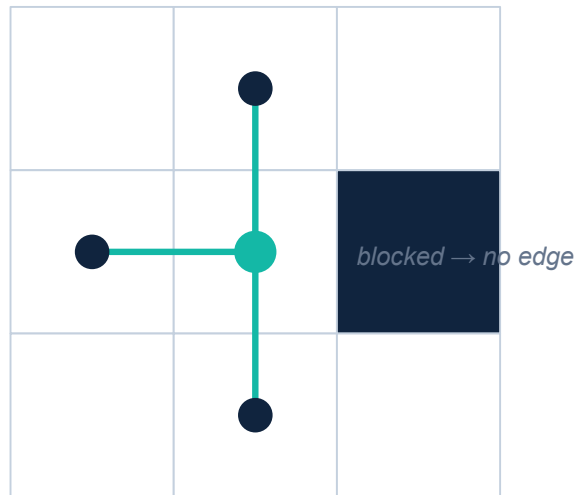
The Grid IS a Graph

Here is the mental shift that unlocks everything: **every free cell is a node**; **every legal step to a free neighbour is an edge**. We never build the graph explicitly — it lives implicitly inside the matrix.

4-connectivity: each cell talks to its 4 cardinal neighbours only.

```
# The 4 possible moves from any cell (x, y)
directions = [(1,0), (-1,0), (0,1), (0,-1)]
#           right  left   down   up
```

On the right: the centre node connects to 4 neighbours. The blocked cell (dark) produces no edge — obstacles simply have no node.





Three Data Structures Every Search Needs

1

Open set (frontier)

Nodes we have discovered but not yet explored. In A*, this is a priority queue ordered by f-score — the most promising node always pops first.

```
open_set = PriorityQueue()
```

2

Came-from map

For each node: which node did we arrive from? This chain of parent links is what lets us rebuild the path once the goal is reached.

```
came_from[(x, y)] = current
```

3

g-score map

The cheapest cost found so far from start to each node. If we later find a cheaper way in, we update it and re-queue the node.

```
g_score[start] = 0
```



The Algorithm Family Tree

Four relatives, one difference: how each one picks the next node to explore.

Algorithm	Picks next node by...	Optimal?	Efficient?
BFS	First-in-first-out queue	Yes (uniform cost)	No — floods equally in all directions
Dijkstra	Lowest actual cost $g(n)$	Yes	Medium — ignores where the goal is
Greedy Best-First	Lowest estimate $h(n)$	No	Fast, but can pick bad paths
A*	Lowest $g(n) + h(n)$	Yes	Yes — best of both worlds



A* is the sweet spot: optimal AND efficient, because it balances what it knows (cost so far) with what it estimates (cost remaining).

The A* Core Formula

$$f(n) = g(n) + h(n)$$

$g(n)$

KNOWN

Actual cost of the cheapest path
found so far from start $\rightarrow n$

$h(n)$

ESTIMATED

Heuristic guess of the remaining
cost from $n \rightarrow$ goal

$f(n)$

DECISION

Estimated total cost of the best
path through n — lowest f pops
next

The priority queue's only job: always hand us the node with the lowest f -score.



The Heuristic: Manhattan Distance

```
def heuristic(cell, goal):  
    return abs(cell[0]-goal[0]) + abs(cell[1]-goal[1])
```

Why Manhattan? With only horizontal/vertical moves you can never reach the goal in fewer than $|\Delta x| + |\Delta y|$ steps. So this estimate **never overestimates** — it is **admissible**.

Why admissibility matters: if $h(n)$ ever overestimates, A^* may skip the optimal path because it looks “too expensive.” Admissible h guarantees the shortest path.

Pick the heuristic that matches the movement

Manhattan $|\Delta x| + |\Delta y|$
4-directional grids

Euclidean $\sqrt{(\Delta x)^2 + (\Delta y)^2}$
diagonal movement

Chebyshev $\max(|\Delta x|, |\Delta y|)$
8-dir, uniform diagonals

$h(n) = 0$ zero everywhere
 A^* becomes Dijkstra



A* Pseudocode — Learn This Shape

INITIALIZE:

```
open_set <- priority queue with (f=0, start)
came_from <- empty dictionary
g_score[all cells] <- infinity ; g_score[start] <- 0
```

LOOP while open_set is not empty:

```
current <- pop node with lowest f-score
if current == goal: RETURN reconstruct_path(...)
```

for each neighbor of current:

```
if neighbor is in-bounds AND free (grid value == 1):
    tentative_g = g_score[current] + 1
    if tentative_g < g_score[neighbor]:      # better path?
        came_from[neighbor] = current
        g_score[neighbor] = tentative_g
        f = tentative_g + heuristic(neighbor, goal)
        add (f, neighbor) to open_set
```

```
RETURN None    # queue exhausted -> no path exists
```

Pop

Cheapest promise first — the priority queue guarantees it.

Relax

Only accept a neighbour if we found a cheaper way in.

Record

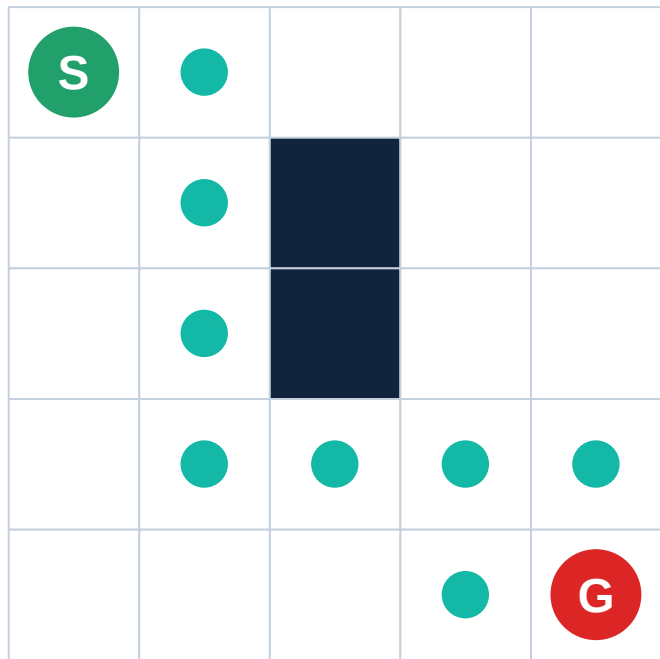
Parent link + new g-score + push with fresh f.

Fail loudly

Empty queue means no path — return None, never loop forever.



Trace It By Hand: 5×5 Example



$S=(0,0)$ $G=(4,4)$ dark = obstacle wall in column 2

Step 1 Start at $S(0,0)$ with $g=0$. Discover neighbours $(1,0)$ and $(0,1)$; both get $g=1$.

Step 2 $f(1,0) = 1 + |1-4| + |0-4| = 8$. $f(0,1)$ is also 8. Tie — pick either.

Step 3+ Keep popping the lowest- f node. The wall in column 2 forces the search around it; far-away nodes collect high f -scores and simply wait in the queue.

Key insight: the heuristic is a compass — nodes pointing away from the goal get higher f and are deprioritised.



The Complete A* (Python)

```
from queue import PriorityQueue
import numpy as np

def a_star_search(occupancy_grid, start, goal):
    def heuristic(cell, goal):
        return abs(cell[0]-goal[0]) + abs(cell[1]-
        goal[1])

    open_set = PriorityQueue()
    open_set.put((0, start))          # (f-score,
    node)
    came_from = {}
    g_score = {c: float('inf')
                for c in
    np.ndindex(occupancy_grid.shape)}
    g_score[start] = 0

    while not open_set.empty():
        _, current = open_set.get()    # lowest f
        pops
        if current == goal:
            return reconstruct_path(came_from, current)
```

```
        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            x, y = current[0]+dx, current[1]+dy

            # bounds check AND obstacle check
            if (0 <= x < occupancy_grid.shape[1] and
                0 <= y < occupancy_grid.shape[0] and
                occupancy_grid[y, x] == 1): # free?

                tentative_g = g_score[current] + 1
                if tentative_g < g_score[(x, y)]:
                    came_from[(x, y)] = current
                    g_score[(x, y)] = tentative_g
                    f = tentative_g + heuristic((x,y),
                    goal)

                    open_set.put((f, (x, y)))

        return None # no path exists
```

Read left column first, then right — it is one function.



Coordinate Convention — The #1 Bug Source

This single mix-up causes more failed submissions at RAIN than everything else combined:

Cells are stored as

(x, y)

x = column, y = row

NumPy indexes as

array[y, x]

row first — y before x!

Free-cell check

```
occupancy_grid[y, x] == 1
```

y comes first

Bounds: columns

```
x < grid.shape[1]
```

shape[1] = number of columns

Bounds: rows

```
y < grid.shape[0]
```

shape[0] = number of rows

If your path goes through walls — this is almost certainly the bug.



Path Reconstruction: Walking Home

Every improvement A* finds records a parent link: `came_from[(x,y)] = current`. The result is a chain: `goal → parent → grandparent → ... → start`. To get the path, walk the chain backwards.

Simple version

```
def reconstruct_path(came_from, current):
    path = [current]          # start at goal
    while current in came_from:
        current = came_from[current]
        path.insert(0, current) # prepend
    return path               # [start, ..., goal]
```

Faster version (large maps)

```
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current) # O(1)
    path.reverse()          # one O(n) pass
    return path
```

Performance note: `path.insert(0, x)` is $O(n)$ per call $\rightarrow O(n^2)$ total. `append` + one reverse is $O(n)$. On a 400×400 map, that difference is real.



Seeing Your Work: Matplotlib

```
import matplotlib.pyplot as plt

plt.imshow(occupancy_grid, cmap='gray', origin='lower')
plt.colorbar(); plt.grid(True)

# Green = Start, Red = Goal
plt.plot(start[0], start[1], 'go', label='Start')
plt.plot(goal[0], goal[1], 'ro', label='Goal')

# Blue = Path
plt.plot([c[0] for c in path],    # x-coordinates
         [c[1] for c in path],    # y-coordinates
         marker='o', markersize=4,
         color='blue', label='Path')

plt.legend(); plt.title("A* Path on Occupancy Grid")
plt.show()
```

origin='lower' is crucial

Without it, row 0 appears at the top (image convention), not the bottom (maths convention). We want y increasing upward.

Why the coordinates align

Our path stores (x, y) tuples and plt.plot takes (x_list, y_list) — so the lists line up naturally with no swapping.



Making It Fast: Two Upgrades

Upgrade 1 — Stop pre-allocating g_score

The dict comprehension creates 160,000 entries for a 400×400 grid before search even begins.

```
from collections import defaultdict

g_score = defaultdict(lambda: float('inf'))
g_score[start] = 0
# entries created only when accessed
```

Upgrade 2 — Add a closed set

Without tracking visited nodes, the same node can be popped and expanded multiple times.

```
closed_set = set()
while not open_set.empty():
    _, current = open_set.get()
    if current in closed_set:
        continue # already explored
    closed_set.add(current)
    # ... rest of loop
```

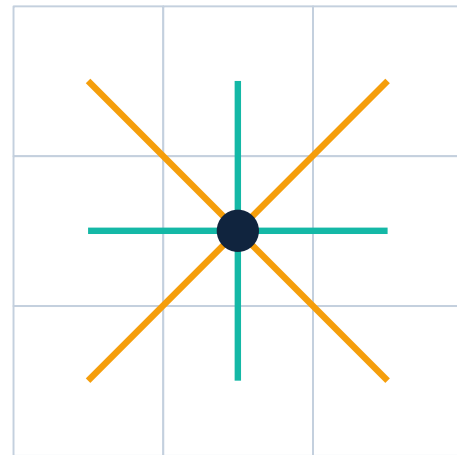
Rule of thumb: measure first. Run both versions on a 400×400 map and time them.



Extension: Diagonal Movement

```
import math
directions = [
    (1,0), (-1,0), (0,1), (0,-1),    # cardinal: cost 1.0
    (1,1), (1,-1), (-1,1), (-1,-1)  # diagonal: cost  $\sqrt{2}$ 
]

for dx, dy in directions:
    step_cost = math.sqrt(dx**2 + dy**2)
    tentative_g = g_score[current] + step_cost
```



teal = cost 1.0 amber = cost ≈ 1.414

Heuristic must change too: with diagonals, Manhattan overestimates and breaks admissibility. Switch to Euclidean or Chebyshev.



Exercise Ladder — Climb During the Week

1

Manual grid **BEGINNER**

Build a 10×10 grid of ones; add walls with $\text{grid}[3, 2:8] = 0$ and $\text{grid}[6, 2:8] = 0$. Run A* from (1,1) to (8,8) and plot the result.

2

BFS vs A* **INTERMEDIATE**

Implement both. Count nodes expanded by each; shade explored regions light blue. A* should visit far fewer nodes — prove it with numbers.

3

Diagonal A* **INTERMEDIATE**

Add 8-directional movement with $\sqrt{2}$ diagonal cost and a Euclidean heuristic. Compare path length and shape against 4-directional.

4

Dynamic re-planning **ADVANCED**

Find a path, then programmatically drop an obstacle onto it. Re-run A*. Plot old path (dashed red) and new path (solid blue) together.

5

Obstacle inflation **ADVANCED**

Real robots have width. Use `scipy.ndimage.binary_dilation` (iterations=5) to inflate obstacles by a safety margin before planning.

Exercises 1–3 are due before Friday's lab. 4–5 earn bonus marks.



Weekend Practical A — Simulation Mission

SOLO OR PAIRS

PYTHON + OPENCV + MATPLOTLIB

SUBMIT: CODE + 60-SEC SCREEN RECORDING

“The Warehouse Delivery Run”

- 1 Draw your own warehouse as a 400×400 image with `cv2.rectangle` shelving blocks (Exercise 6 code from the handout). Save as `my_grid.png`.
- 2 Convert it to an occupancy grid with your thresholding function, then inflate obstacles by 5 cells — your “robot” has a body.
- 3 Run A* between 3 pickup → drop-off pairs of your choice. Plot each route over the map with start/goal markers and a legend.
- 4 Mid-mission surprise: after the first route is found, drop a new obstacle across it and re-plan live. Show old (dashed red) vs new (solid blue).
- 5 Report: total path length, nodes expanded, and planning time for each run — printed neatly to the console.

Stretch goal: animate the robot dot moving cell-by-cell along the path with `plt.pause(0.05)`.



Weekend Practical B — Physical Build

TEAMS OF 3

ARDUINO + MOTOR DRIVER + ULTRASONIC

RAIN LAB: TAPE, CARTONS, CHASSIS KITS

“Grid Runner” — drive the path your planner computed

- 1 Tape a 6×6 grid on the lab floor (40 cm cells). Place carton “shelves” on 6–8 cells and photograph the arena from directly above.
- 2 Convert your photo to an occupancy grid (threshold + resize to 6×6). Verify the matrix matches reality — fix the threshold until it does.
- 3 Run A* on your 6×6 matrix, then convert the path into motion commands: F (forward one cell), L / R (turn 90°).
- 4 Program the Arduino robot to execute the command string using timed moves. Calibrate: how many milliseconds = one 40 cm cell?
- 5 Safety layer: if the ultrasonic sensor sees an obstacle < 15 cm mid-run, STOP — someone moved a box. (Bonus: re-photograph and re-plan.)

Deliverable: 2-minute video — arena photo, planned path plot, robot completing the run. Demo day: Monday 9 AM.



✓ Key Takeaways

- ✓ Occupancy grids turn images into binary matrices a computer can reason about: 1 = free, 0 = blocked.
- ✓ The priority queue guarantees we always expand the most promising node next.
- ✓ Thresholding is the bridge — it decides the cutoff between free and occupied.
- ✓ Path reconstruction walks the `came_from` chain backwards, goal to start.
- ✓ A* evaluates $f(n) = g(n) + h(n)$: known cost so far plus estimated cost remaining.
- ✓ Coordinate conventions — (x, y) vs [row, col] — are the most common source of bugs.
- ✓ Manhattan distance is the right heuristic for 4-directional grids because it is admissible.

The concept chain to master:

Image → Threshold → Occupancy Grid → Implicit Graph → Search (BFS → Dijkstra → A*) → Heuristics → Priority Queue → Reconstruction → Visualization